

Optimisation combinatoire
2^e année du cycle supérieur — Spécialité SIQ

Problème de Bin Packing 1D

Méthodes approchées : Métaheuristiques de voisinage

Réalisé par

BOUKAKIOU Rayan
KHERROUBI Mohamed El Amine
ZIADI Idriss Yacine
HIMEUR Idris
DIAR Adem Abdelhafidh

1 Introduction

Le problème de **Bin Packing 1D** consiste à minimiser le nombre de bacs de capacité C nécessaires pour ranger n objets de tailles w_i . Après avoir étudié les heuristiques de construction (FFD, BFD) et d'amélioration locale (Relocation, Swap), nous explorons ici les **métaheuristiques**. Contrairement aux méthodes locales simples, ces approches permettent de sortir des optima locaux en acceptant temporairement des dégradations de la solution.

2 Formulation du problème

On dispose d'objets $i \in \{1, \dots, n\}$ de tailles $w_i \in \mathbb{N}$ et de bacs de capacité C . L'objectif est de **minimiser le nombre de bacs utilisés**, avec $x_{ij}, y_j \in \{0, 1\}$:

$$\min \sum_{j=1}^n y_j \quad \text{s.c.} \quad \sum_j x_{ij} = 1 \quad \forall i, \quad \sum_i w_i x_{ij} \leq C y_j \quad \forall j.$$

3 Modélisation et Voisinage

L'évaluation d'une solution ne repose pas uniquement sur le nombre de bacs utilisés. En effet, une telle approche engendre de vastes plateaux dans l'espace de recherche, rendant l'exploration inefficace. Pour pallier ce problème, nous adoptons une fonction de coût qui pénalise les espaces inutilisés afin de favoriser un meilleur compactage des objets :

$$\text{Cost}(S) = |S| \times 10000 + \sum_{b \in S} (C - \text{load}(b))^2$$

Cette fonction combine une forte pénalisation du nombre de bacs avec une mesure quadratique du vide restant dans chaque bac.

Le voisinage exploité par les deux métaheuristiques repose sur deux types de mouvements principaux :

- **Relocation** : transfert d'un objet depuis un bac source vers un bac destination.
- **Swap** : échange de deux objets appartenant à deux bacs distincts.

Enfin, la solution initiale est générée à l'aide d'une heuristique de construction basée sur un tri décroissant des objets, similaire à l'algorithme *First Fit Decreasing* (FFD), ce qui permet de démarrer la recherche à partir d'une solution de bonne qualité.

4 Métaheuristiques de voisinage

4.1 Recuit Simulé (Simulated Annealing - SA)

Le recuit simulé est une métaheuristique qui autorise, avec une certaine probabilité, l'acceptation de solutions moins bonnes que la solution courante. Cette probabilité dépend à la fois de l'importance de la dégradation et d'un paramètre appelé température T , qui décroît progressivement au cours de l'exécution.

Variantes implémentées et paramètres étudiés :

- **SA classique (default)** : Utilise un schéma de refroidissement géométrique standard, caractérisé par un taux **cooling_rate=0.995**, avec un nombre maximal de **25 000 itérations**.
- **SA avec réchauffement (reheating)** : En cas de stagnation (absence d'amélioration pendant 600 itérations), la température est augmentée à nouveau selon un facteur **reheating_factor=0.45**. Cette stratégie permet de diversifier la recherche et de s'extraire de minima locaux.

- **SA adaptatif (adaptive)** : La température est ajustée dynamiquement toutes les **200 itérations** afin de maintenir un taux d'acceptation proche de 30 %. Cette adaptation vise à équilibrer exploration et exploitation tout au long de la recherche.

Algorithme 1: Recuit Simulé Adaptatif / avec Réchauffement

Input: Solution initiale S_0 , $T_{initial}$, cooling_rate α

Output: Meilleure solution S_{best}

```

1  $S \leftarrow S_0$ ;
2  $S_{best} \leftarrow S_0$ ;
3  $T \leftarrow T_{initial}$ ;
4 pour chaque iteration de 1 to  $Iter_{max}$  faire
5    $S' \leftarrow \text{Générer\_Voisin\_Aleatoire}(S)$ ;
6    $\Delta \leftarrow \text{Cost}(S') - \text{Cost}(S)$ ;
7   si  $\Delta \leq 0$  ou  $\text{random}() < e^{-\Delta/T}$  alors
8      $S \leftarrow S'$ ;
9     si  $\text{Cost}(S) < \text{Cost}(S_{best})$  alors
10     $S_{best} \leftarrow S$ ;
11   Mettre à jour  $T$  (Refroidissement :  $T \leftarrow T \times \alpha$ );
12   si stagnation et variante_reheating alors
13      $T \leftarrow T \times \text{facteur\_réchauffement}$ ;
14   si variante_adaptive alors
15     ajuster  $T$  selon taux d'acceptation;
16 return  $S_{best}$ ;

```

4.2 Recherche avec Tabou (Tabu Search - TS)

La recherche tabou parcourt le voisinage en sélectionnant le meilleur mouvement admissible. Afin d'éviter les cycles, les mouvements récents sont placés dans une liste tabou et interdits pendant un certain nombre d'itérations (tenure).

Variantes implémentées et paramètres étudiés :

- **TS Classique (ts)** : tenure tabou fixe, proportionnelle au nombre d'objets.
- **TS Réactif (rts)** : la taille de la liste tabou (tabu_tenure) s'adapte dynamiquement, elle augmente en cas de cycles (détectés via la signature des solutions) ou de stagnation (jusqu'à 300 itérations), et diminue lors d'une amélioration.
- **TS avec LNS (Ints)** : intègre une Large Neighborhood Search. Périodiquement (lns_period=450) ou en cas de stagnation prolongée (260 itérations), 12 % des objets sont retirés des bacs peu chargés (phase destroy) puis réinsérés heuristiquement (repair) afin de diversifier la solution.
- **TS Diversifié (hybrid tabu)** : introduit une mémoire à long terme en pénalisant les mouvements les plus fréquents (**move_frequency**).

Algorithme 2: Recherche avec Tabou (Réactif / LNS / Diversifié)

Input: Solution initiale S_0 , $Iter_{max}$, tabu_tenure initiale**Output:** Meilleure solution S_{best}

```
1  $S \leftarrow S_0$ ;  
2  $S_{best} \leftarrow S_0$ ;  
3  $ListeTabou \leftarrow \emptyset$ ;  
4  $Tenure \leftarrow tabu\_tenure$ ;  
5 pour chaque iteration de 1 to  $Iter_{max}$  faire  
6   si variante_lns et (stagnation prolongée ou période_LNS atteinte) alors  
7      $S \leftarrow Destroy\_and\_Repair(S, 12\% \text{ des objets})$  ; // Perturbation majeure  
8     Mettre à jour la signature de hachage de la solution;  
9   sinon  
10     $Voisins \leftarrow Générer\_Candidats(S, max\_candidates)$ ;  
11    si variante_diversified alors  
12      Pénaliser l'évaluation des candidats dans  $Voisins$  selon leur move_frequency;  
13     $S' \leftarrow$  Meilleur candidat  $\in Voisins$  tel que (mouvement  $\notin ListeTabou$ ) ou (Critère  
14      d'aspiration :  $Cost(S') < Cost(S_{best})$ );  
15     $S \leftarrow S'$ ;  
16    Ajouter le mouvement inverse dans  $ListeTabou$  pour  $Tenure$  itérations;  
17  si  $Cost(S) < Cost(S_{best})$  alors  
18     $S_{best} \leftarrow S$ ;  
19    si variante_reactive alors  
20       $Tenure \leftarrow \max(Tenure_{min}, Tenure - \text{décrément})$ ;  
21  sinon si variante_reactive et cycle_déecté(S) alors  
22     $Tenure \leftarrow \min(Tenure_{max}, Tenure \times \text{facteur\_augmentation})$ ;  
23 return  $S_{best}$ ;
```

5 Réglage des paramètres et résultats

5.1 Méthodologie de sélection des paramètres (Tuning)

Pour éviter un choix arbitraire des paramètres, nous avons utilisé une analyse de sensibilité.

1. **Jeu de test** : Utilisation d'un ensemble mixte d'instances (Scholl et générées) avec une difficulté progressive (MAX_ITEMS de 100 à 500).
2. **Approche OAT (One-at-a-time)** : Pour chaque métaheuristique, un seul paramètre est modifié à la fois tandis que les autres restent fixes, afin d'observer son effet.
3. **Métriques utilisées** :
 - **bins_mean** : critère principal (minimiser le nombre de bacs).
 - **score_mean** : permet de comparer des solutions ayant le même nombre de bacs en évaluant le compactage.
 - **time_mean** : mesure le temps d'exécution.

5.2 Analyse de sensibilité du Recuit Simulé (SA)

Paramètre	Plage (Valeur)	Impact	Rôle
<code>cooling_rate</code>	0.85 à 0.999 (0.995)	Critique	Contrôle la vitesse de convergence. Un taux de 0.995 offre le meilleur compromis.
<code>initial_temp</code>	1.0 à 1000.0 (100.0)	Moyen	Définit la probabilité initiale d'accepter de mauvaises solutions. Essentiel pour sortir de l'optimum FFD initial.
<code>target_acceptance</code>	0.1 à 0.6 (0.3)	Élevé	(Variante Adaptative) Ajuste T pour maintenir 30% d'acceptation. Évite une descente locale prématurée.
<code>adaptive_window</code>	Fixé à 200	Faible	Fréquence de mise à jour de la température. Une fenêtre trop courte rend T instable.
<code>stagnation_limit</code>	200 à 1000 (600)	Moyen	(Variante Reheating) Nombre d'itérations sans amélioration avant de réchauffer.
<code>reheating_factor</code>	Fixé à 0.45	Moyen	Intensité du choc thermique lors du réchauffement pour relancer l'exploration.
<code>max_iterations</code>	25 000	Proportionnel	Définit le temps de calcul. Le score se stabilise généralement après 15 000 itérations.

Le paramètre le plus sensible reste le **cooling_rate**. Cependant, l'activation de **use_adaptive** rend l'algorithme beaucoup moins dépendant du choix arbitraire de la température initiale, car il s'auto-calibre selon le relief de l'instance.

5.3 Analyse de sensibilité de la Recherche Tabou (TS)

Paramètre	Plage (Valeur)	Impact	Rôle
tabu_tenure_ratio	0.05 à 0.2 (0.1)	Élevé	Définit la durée d'interdiction d'un mouvement ($10\% \times N$). Crucial pour éviter les cycles de retour.
max_candidates	5% à 30% (15%)	Élevé	Proportion du voisinage explorée. Au-delà de 15%, le gain ne justifie plus le temps de calcul.
patience_ratio	0.02 à 0.1 (0.05)	Critique	Seuil de stagnation avant de déclencher le LNS ou le mode Réactif. Déterminant pour la robustesse.
diversification	0.0 à 5.0 (2.0)	Moyen	Poids de la pénalité de fréquence. Pousse l'algorithme vers des régions rarement explorées.
lns_size_ratio	Fixé à 0.12	Élevé	Pourcentage d'objets retirés lors d'un Destroy. Permet un réarrangement majeur.
lns_period	Fixé à 450	Moyen	Fréquence cyclique de perturbation majeure. Complète la patience en forçant l'exploration.
stagnation_limit	Fixé à 260	Moyen	Seuil spécifique de bascule en mode intensif ou réactif.

Le tuner met en évidence que sans une patience bien réglée, la Recherche Tabou classique reste souvent piégée dans des optima locaux de haute qualité mais non globaux. Le couplage entre la tabu_tenure dynamique (Reactive TS) et les sauts du LNS est la configuration qui minimise le plus systématiquement le nombre de bacs sur les instances de Scholl.

6 Conclusion

Les métaheuristiques de voisinage (recuit simulé et recherche tabou) permettent d'obtenir de meilleures solutions pour le Bin Packing 1D que les heuristiques classiques, grâce à une exploration plus efficace de l'espace de recherche.

D'après nos expérimentations, le recuit simulé adaptatif offre le meilleur compromis entre qualité et temps d'exécution, tandis que la recherche tabou hybride (RTS + LNS) est la plus performante pour réduire le nombre de bacs sur des instances difficiles.

Ce TP met en évidence l'intérêt d'utiliser des approches avancées pour traiter efficacement ce type de problème d'optimisation.